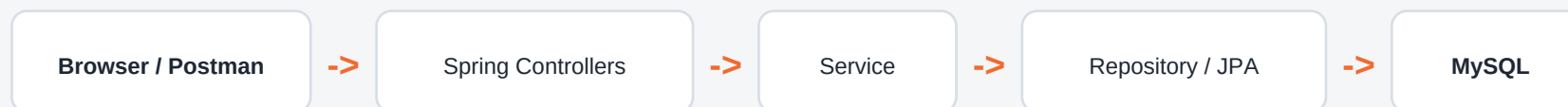


SPRING BOOT MVC, MYSQL & POSTMAN PRACTICAL

Execution Report

Spring Boot MVC + REST API + Spring Data JPA + Hibernate + MySQL + Postman CRUD



Prepared by

K. Pranav | Roll No: 2520030477

Evidence pages contain the successful execution screenshots captured from Spring Tool Suite, browser, Postman and MySQL.

01

Spring Tool Suite - Project & Successful Startup

The Spring Boot project structure is shown together with a successful JPA, Hibernate, MySQL and Tomcat startup.

The screenshot displays the Spring Tool Suite IDE interface. On the left, the Package Explorer shows the project structure for 'student-rest-api'. The main editor shows the code for 'StudentRestController.java'. The bottom console window displays the startup logs for the application.

Package Explorer:

- student-rest-api [boot] [devtools]
 - src/main/java
 - com.example.demo
 - StudentRestApiApplication.java
 - com.example.demo.controller
 - StudentController.java
 - StudentRestController.java
 - com.example.demo.model
 - Student.java
 - com.example.demo.repository
 - StudentRepository.java
 - com.example.demo.service
 - StudentService.java
 - src/main/resources
 - static
 - templates
 - application.properties
 - src/test/java
 - JRE System Library [JavaSE-17]
 - Maven Dependencies
 - src
 - target

Code Editor:

```

57  @RequestBody Student student) {
58
59      Student updatedStudent =
60          studentService.updateStudent(id, student);
61
62      if (updatedStudent == null) {
63          return ResponseEntity.notFound().build();
64      }
65
66      return ResponseEntity.ok(updatedStudent);
  
```

Console Output:

```

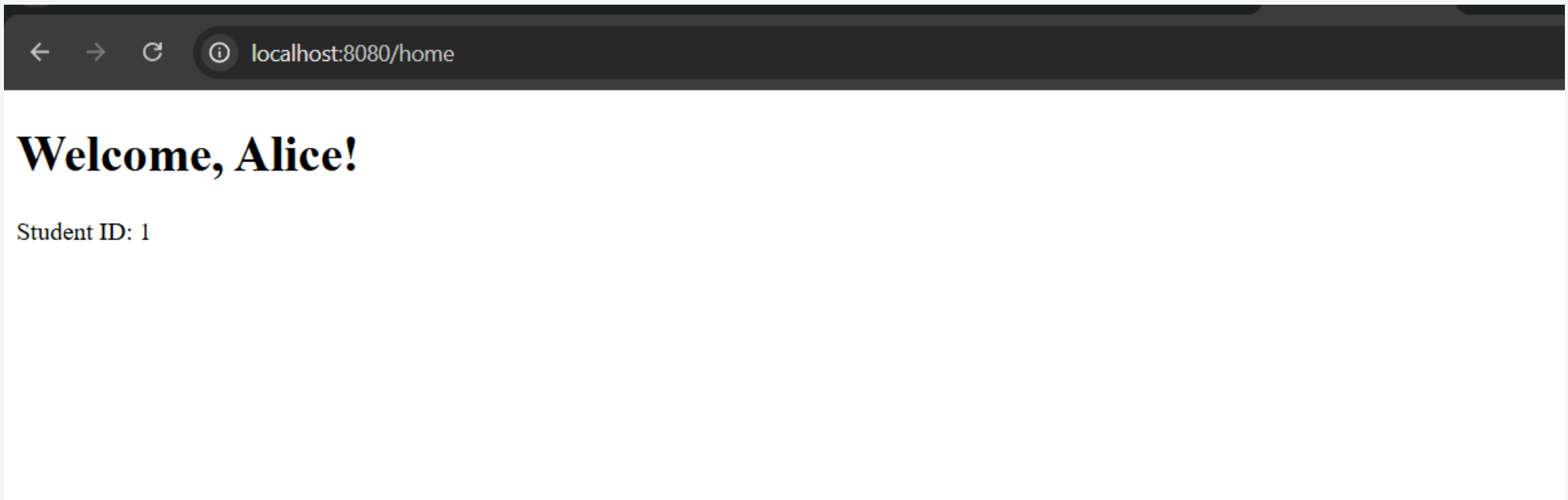
2026-09-03T08:43:50.554+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] c.e.demo.StudentRestApiApplication : Starting StudentRestApiApplicat
2026-09-03T08:43:50.558+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] c.e.demo.StudentRestApiApplication : No active profile set, falling
2026-09-03T08:43:50.629+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults acti
2026-09-03T08:43:50.629+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logg
2026-09-03T08:43:51.805+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA r
2026-09-03T08:43:51.884+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository
2026-09-03T08:43:52.505+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 86
2026-09-03T08:43:52.522+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-09-03T08:43:52.524+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apach
2026-09-03T08:43:52.573+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: ini
2026-09-03T08:43:52.789+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] org.hibernate.orm.jpa : Starting service [Tomcat]
2026-09-03T08:43:52.887+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] org.hibernate.orm.core : HHH000001: Hibernate ORM core v
2026-09-03T08:43:53.523+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup; ignori
2026-09-03T08:43:53.567+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-09-03T08:43:54.101+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connector
2026-09-03T08:43:54.105+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-09-03T08:43:54.256+05:30 INFO 30116 --- [student-rest-api] [ restartedMain] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
  
```

Database JDBC URL [jdbc:mysql://localhost:3306/student_api_db]

02

MVC Output - Home Page

The /home endpoint renders the Student model through a Thymeleaf view and displays the configured student details.



03

MVC Output - Request Parameter

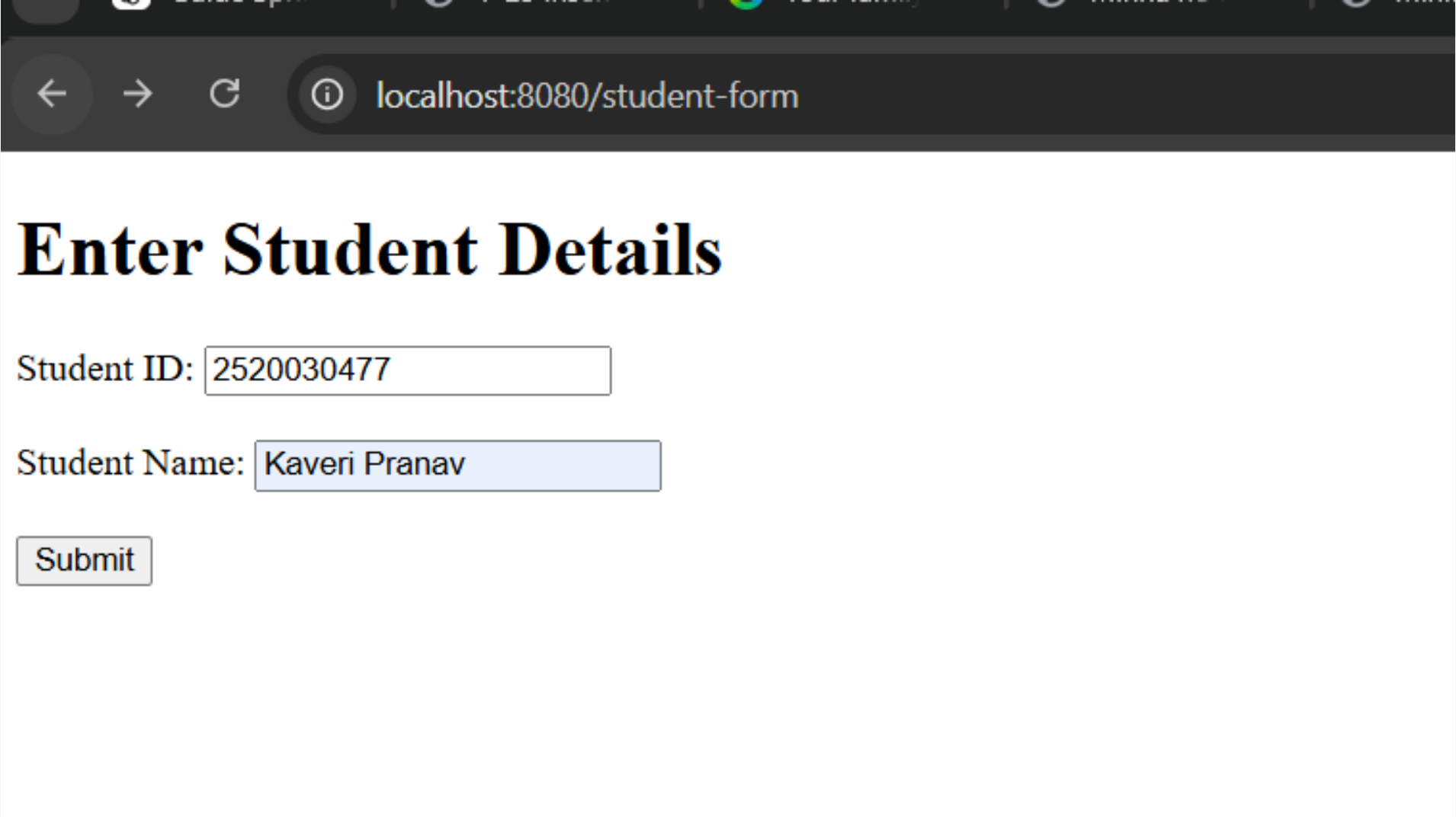
The /hello endpoint reads the name query parameter and renders the personalized Thymeleaf response.



04

MVC Student Form

The student form accepts a student ID and name and submits the values to the Spring MVC controller.



The screenshot shows a web browser window with the address bar displaying `localhost:8080/student-form`. The page content includes a heading **Enter Student Details**, followed by two input fields: **Student ID:** with the value `2520030477` and **Student Name:** with the value `Kaveri Pranav`. A **Submit** button is located below the input fields.

05

MVC Student Form - Result

After form submission, Spring binds the submitted data to the Student model and displays the result page.

Student Details

ID: 2147483647

Name: Kaveri Pranav

06

Postman POST - Create Student

200 OK

A POST request sends JSON to the REST API and successfully creates a new student record.

The screenshot displays the Postman application interface. On the left, the 'COLLECTIONS' sidebar shows 'My Collection' with two items: 'GET Get data' and 'POST Post data'. The main workspace is configured for a POST request to 'http://localhost:8080/api/students'. The request body is a JSON object:

```
{  "name": "Pranav"}
```

. The response status is '200 OK' with a response time of 552 ms and a size of 178 B. The response body is a JSON object:

```
{  "id": 4,  "name": "Pranav"}
```

. On the right, the 'New Chat' panel shows a message from an AI assistant: 'Update the status code assertion to expect 400 instead of 200, as the request is missing a required body.' The chat also displays a test script snippet:

```
pm.test("Status code is 200", function () {  // ...  pm.test("Status code is 400", function () {  // ...
```

07

MySQL Verification - Created Record

The student created through Postman is verified directly in the MySQL student table.

```
mysql> SELECT * FROM student;
+----+-----+
| id | name                |
+----+-----+
| 1  | Add your name in the body |
| 2  | Add your name in the body |
| 4  | Pranav              |
+----+-----+
3 rows in set (0.00 sec)
```

08

Postman GET - Read All Students

200 OK

A GET request retrieves all student records stored in MySQL through the Spring Boot REST API.

The screenshot displays the Postman application interface. On the left, the 'COLLECTIONS' sidebar shows 'My Collection' with two items: 'GET Get data' (selected) and 'POST Post data'. The main workspace shows a GET request to 'http://localhost:8080/api/students'. The 'Body' tab is active, displaying a JSON array of student records. The response status is '200 OK' with a response time of 522 ms and a size of 269 B. The response body is a JSON array containing three student records. On the right, the 'New Chat' panel is open, showing a message from the AI assistant: 'Update the status code assertion to expect 400 instead of 200, as the request is missing a required body.' Below this, there are suggestions to 'Fix the 400 by improving tests', 'Add required request body fields', and 'Parameterize the URL and method'. The bottom status bar shows 'Fix status code assertion'.

```
GET http://localhost:8080/api/students
```

```
{
  "name": "Pranav"
}
```

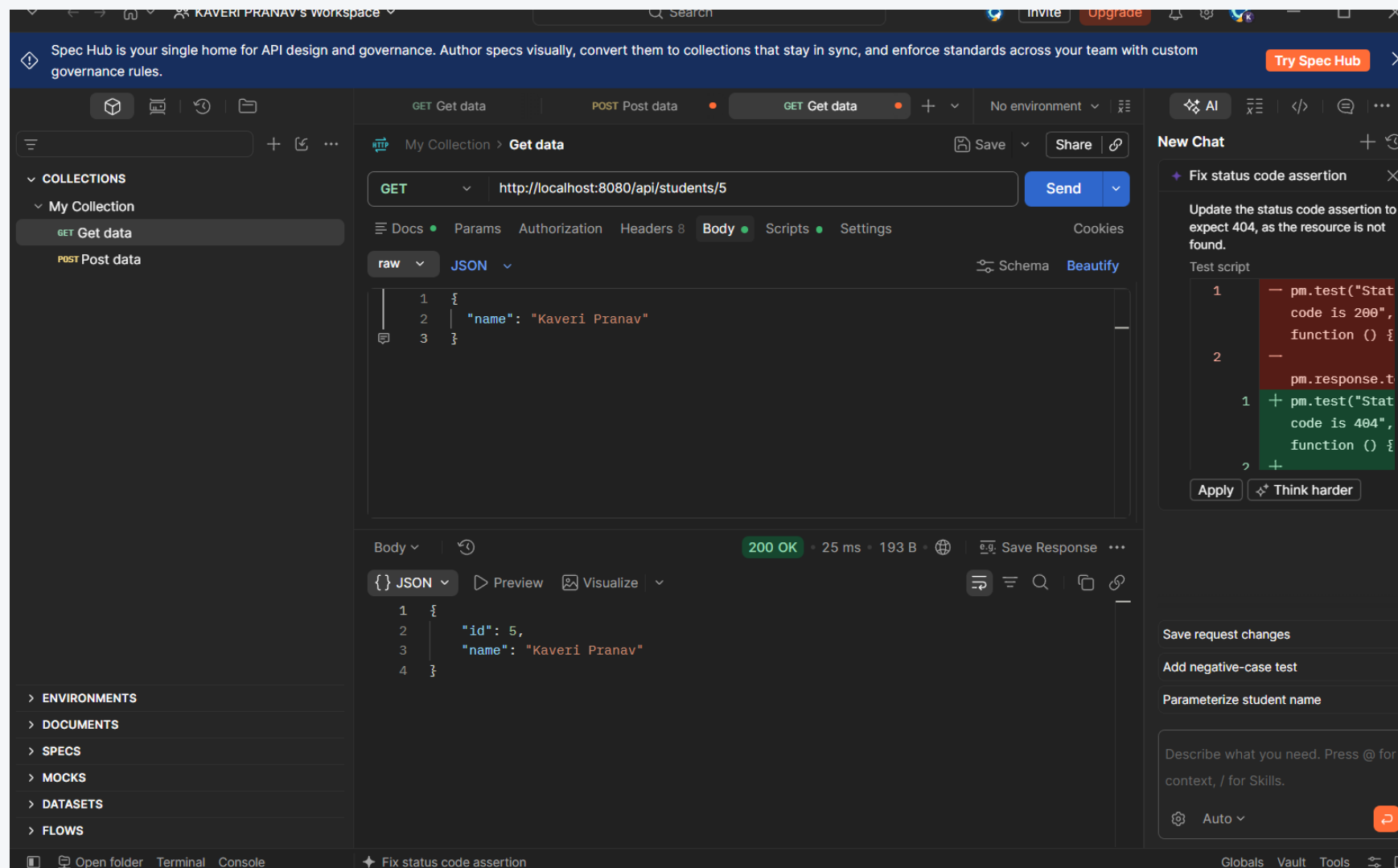
```
[
  {
    "id": 1,
    "name": "Add your name in the body"
  },
  {
    "id": 2,
    "name": "Add your name in the body"
  },
  {
    "id": 4,
    "name": "Pranav"
  }
]
```

09

Postman GET by ID

200 OK

A GET request using `/api/students/{id}` retrieves one specific student record by its ID.



10

Postman PUT - Update Student

200 OK

A PUT request updates the selected student record and returns the modified student as JSON.

The screenshot displays the Postman application interface. At the top, a banner for 'Spec Hub' is visible. The left sidebar shows a 'COLLECTIONS' list with 'My Collection' expanded, containing 'GET Get data' and 'POST Post data'. The main workspace is configured for a 'PUT' request to 'http://localhost:8080/api/students/5'. The 'Body' tab is selected, showing a raw JSON payload:

```
{  "name": "K Pranav"}
```

. Below the request, the response is shown as a 200 OK status with a 62 ms response time and 188 B of data. The response body is a JSON object:

```
{  "id": 5,  "name": "K Pranav"}
```

. On the right, a 'New Chat' panel is open, displaying a message from an AI assistant: 'Update the status code assertion to expect 404, as the resource is not found.' The chat panel also shows a 'Test script' with a suggestion to change the status code assertion from 200 to 404. At the bottom, a 'Fix status code assertion' button is visible.

11

MySQL Verification - Updated Record

The updated student name is verified directly in MySQL after the PUT request.

```
mysql> SELECT * FROM student;
+----+-----+
| id | name                |
+----+-----+
|  1 | Add your name in the body |
|  2 | Add your name in the body |
|  5 | K Pranav            |
+----+-----+
3 rows in set (0.00 sec)

mysql> |
```

12

Postman DELETE - Delete Student

204 SUCCESS

A DELETE request removes the selected student and returns HTTP 204 No Content on success.

Spec Hub is your single home for API design and governance. Author specs visually, convert them to collections that stay in sync, and enforce standards across your team with custom governance rules. [Try Spec Hub](#)

GET Get data | POST Post data | **DEL Get data** | No environment | [AI](#) | [x](#) | [</>](#) | [@](#) | [...](#)

My Collection > **Get data** | [Save](#) | [Share](#)

DELETE | [http://localhost:8080/api/students/4](#) | [Send](#)

[Docs](#) | [Params](#) | [Authorization](#) | [Headers 8](#) | **Body** | [Scripts](#) | [Settings](#) | [Cookies](#)

raw | **JSON** | [Schema](#) | [Beautify](#)

```
1 {
2   "name": "Pranav"
3 }
```

Body | [Refresh](#) | **204 No Content** | 1.01 s | 102 B | [Save Response](#) | [...](#)

[Raw](#) | [Preview](#) | [Visualize](#) | [...](#)

1

[Fix the status code assertion](#)

Update the status code assertion to expect 204 No Content, which is the correct status for this DELETE request.

Test script

```
1 - pm.test("Stat
code is 200",
function () {

2 -
pm.response.t

1 + pm.test("Stat
code is 204",
function () {

2 +
```

[Apply](#) | [Think harder](#)

[Fix status-code test expectation](#)

[Add a negative-case assertion](#)

[Create a follow-up request to verify deletion](#)

Describe what you need. Press @ for context, / for Skills.

[Auto](#) | [Refresh](#)

[Open folder](#) | [Terminal](#) | [Console](#) | [Fix the status code assertion](#) | [Globals](#) | [Vault](#) | [Tools](#) | [...](#)

13

MySQL Final Verification - Record Deleted

The final MySQL query confirms that the deleted student record is no longer present in the table.

```
mysql> SELECT * FROM student;
+----+-----+
| id | name                |
+----+-----+
|  1 | Add your name in the body |
|  2 | Add your name in the body |
+----+-----+
2 rows in set (0.00 sec)
```